

Programski alati u fizici

Simulacija dvostrukog njihala

Pavla Rapić

Split, 25.6.2026.

Sažetak

Cilj rada je opisati i analizirati problem dvostrukog njihala kroz modernu geometrijsku mehaniku, linearnu algebru i Lagrangeovu mehaniku. U izradi računalne simulacije korištene su programske biblioteke *NumPy* i *Matplotlib* u jeziku Python uz primjenu numeričke metode Runge-Kutta četvrtog reda (RK4). Uz napomenu da se sama simulacija odvija sporije nego što je fizikalno predviđeno pokazalo se da numerički algoritam svakako replicira fizikalnu stvarnost uz strogo očuvanje ukupne energije. Ako bismo pak inzistirali na iznimnoj preciznosti, onda se uvode druge metode poput simplektičkog integratora (*ovakav numerički algoritam se najčešće koristi u Hamiltonovoj mehanici. Međutim, kako su Hamiltonova i Lagrangeova mehanika slične, takav algoritam se može primijeniti i u Lagrangeovoj.*) koji je izostavljen iz razmatranja kako bi rad bio u skladu sa znanjem stečenim do sada.

1 Uvod

Dvostruko njihalo predstavlja klasični primjer nelinearnog i kaotičnog gibanja. Sastoji se od dvaju štapa i dviju masa pri čemu je prva vezana za površinu, a druga je vezana za nju.

Naime, sama Newtonova mehanika u ovakvom problemu ne igra adekvatnu ulogu pri rješavanju problema jer se sile napetosti niti neprestano i nelinearno mijenjaju, već su nam potrebne Lagrangeove jednačbe kako bismo pojednostavnili problem.

Međutim, rješavanje takvih jednačbi *ab ovo* je složenije i nije elegantan način rješavanja, odnosno implementacije. Stoga u ovome radu umjesto klasičnog rješavanja diferencijalnih jednačbi i postavljanja uvjeta za kutove, uvodimo geometrijsku reprezentaciju te kvadratnu formu pa se ovaj problem rješava direktnom primjenom matričnog računa.

1.1 Konfiguracijski prostor

Budući da se problem rješava geometrijskim pristupom, onda je prirodno zapitati se kako uopće uvesti geometriju. Ako njihalo promatramo na način da je osim niti i mase određeno kutom s obzirom na referentnu os, onda možemo kazati da je svaka točka odnosno trajektorija element kružnice S^1 [1]. Sada kada uzmemo Kartezijev produkt istih, dobivamo konfiguracijski prostor $\mathbb{T}^2$ [1] koji ukratko predstavlja skup svih mogućih

pozicija dvostrukog njihala na kontinuiranoj glatkoj površini torusa poznat kao oblik krafne. Ovaj pristup nam omogućuje da složena geometrijska ograničenja, poput fiksnih duljina niti, ugradimo izravno u sam prostor stanja umjesto da pratimo kretanje u trodimenzionalnom prostoru $\mathbb{R}^3$ uz uvođenje dodatnih jednadžbi veza [1]. Ovo pojednostavljuje stvar prilikom numeričke integracije jer umjesto da uvodimo da su kutovi striktno omeđeni unutar intervala $[0, 2\pi)$, točka se kontinuirano i glatko giba po površini. Dobro je napomenuti da postoji takozvana matrica rotacije koja predstavlja rotaciju u dvodimenzionalnom sustavu:

$$\mathbf{R}(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \quad (1)$$

Stoga ovako postavljen problem eliminira potrebu da konstruiramo program tako da računalu kada dođe do kuta od 2π mora ručno shvaćati da je ponovno na početku jer linearna transformacija rotacije je u kontinuitetu pa se tako problem kuta rješava automatski bez numeričkih skokova. Na ovaj način vektor položaja, odnosno stanje sustava, umjesto nezgrapnih prostornih Kartezijevih parova koordinata (x, y) postaje elegantni vektor poopćenih koordinata $\mathbf{q} = [\theta_1, \theta_2]^T$.

1.2 Lagrangeova mehanika

Lagrangeova mehanika temeljena je na formuli:

$$L = T - V \quad (2)$$

gdje je T kinetička energija, a V potencijalna energija sustava. U geometrijskom pristupu, analiza dinamičkog ponašanja sustava prebacuje se u analizu geodezijskog toka u odgovarajućem Riemannovom prostoru, gdje funkcija Lagrangeijana igra ključnu ulogu u samoj geometrizaciji dinamike [2]. Na ovaj način, rješavajući jednadžbe kroz poopćene koordinate unutar definiranog prostora [3], potpuno eliminiramo sile napetosti niti i problem promatramo isključivo kroz energiju sustava.

1.3 Kvadratna forma

Kada općenito pogledamo u izraz za kinetičku energiju lako se pokaže da je ona ujedno i kvadratna forma po poopćenim brzinama $\dot{\mathbf{q}} = [\dot{\theta}_1, \dot{\theta}_2]^T$. Ako smo već uzeli da nam je vektor stanja dvostrukog njihala $\mathbf{q} = [\theta_1, \theta_2]^T$, onda u općenitom zapisu kinetičku energiju kao kvadratnu formu možemo prikazati u obliku simetričnog matričnog "sendviča" [3, 2]:

$$T = \frac{1}{2} \dot{\mathbf{q}}^T \mathbf{M}(\mathbf{q}) \dot{\mathbf{q}} \quad (3)$$

Kod sustava s niskim brojem stupnjeva slobode gibanja koji imaju nedijagonalni metrički tenzor, poput dvostrukog fizikalnog njihala, dinamika se analizira u sklopu Riemannove geometrije pri čemu kvadratna matrica inercije $\mathbf{M}(\mathbf{q})$ preuzima ulogu metričkog tenzora (Riemannove metrike) [2]. Kada u potpunosti raspišemo ovaj matrični umnožak za dvostruko njihalo, dobivamo sljedeći izraz:

$$T = \frac{1}{2}(m_1 + m_2)L_1^2\dot{\theta}_1^2 + m_2L_1L_2\cos(\theta_1 - \theta_2)\dot{\theta}_1\dot{\theta}_2 + \frac{1}{2}m_2L_2^2\dot{\theta}_2^2 \quad (4)$$

Iz ovog raspisa izravno dobivamo komponente matrice inercije $\mathbf{M}(\mathbf{q})$ reda 2×2 :

$$\mathbf{M}(\mathbf{q}) = \begin{bmatrix} (m_1 + m_2)L_1^2 & m_2L_1L_2 \cos(\theta_1 - \theta_2) \\ m_2L_1L_2 \cos(\theta_1 - \theta_2) & m_2L_2^2 \end{bmatrix} \quad (5)$$

Vrijednosti elemenata ove matrice ovise o trenutnoj poziciji sustava $\mathbf{q}$, a složenost nelinearnih sprega u gibanju krakova očituje se upravo kroz izvandijagonalne elemente koji sadrže kosinus razlike kutova $\cos(\theta_1 - \theta_2)$ [2]. Potencijalna energija sustava ovisi isključivo o visinama unutar gravitacijskog polja i glasi:

$$V = -(m_1 + m_2)gL_1 \cos \theta_1 - m_2gL_2 \cos \theta_2 \quad (6)$$

2 Diskusija i rezultati

Primjenjujući sve napisano, računalna implementacija problema u programskom kodu rješava nelinearni sustav diferencijalnih jednačbi u kompaktnom matričnom obliku:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} = \mathbf{F}_{sile}(\mathbf{q}, \dot{\mathbf{q}}) \quad (7)$$

```

10 def so2_matrix(theta):
11     return np.array([[np.cos(theta), -np.sin(theta)],
12                      [np.sin(theta),  np.cos(theta)]])
13
14 def equations_of_motion(y):
15     t1, t2, w1, w2 = y
16     delta = t1 - t2
17
18     M11 = (m1 + m2) * L1**2
19     M12 = m2 * L1 * L2 * np.cos(delta)
20     M21 = M12
21     M22 = m2 * L2**2
22     M = np.array([[M11, M12],
23                  [M21, M22]])
24
25     C1 = -m2 * L1 * L2 * w2**2 * np.sin(delta) - (m1 + m2) * g * L1 * np.sin(t1)
26     C2 = m2 * L1 * L2 * w1**2 * np.sin(delta) - m2 * g * L2 * np.sin(t2)
27     C = np.array([C1, C2])
28
29     accel = np.linalg.solve(M, C)
30
31     return np.array([w1, w2, accel[0], accel[1]])
32
33 def rk4_step(y, dt):
34     k1 = equations_of_motion(y)
35     k2 = equations_of_motion(y + 0.5 * dt * k1)
36     k3 = equations_of_motion(y + 0.5 * dt * k2)
37     k4 = equations_of_motion(y + dt * k3)
38
39     return y + (dt / 6.0) * (k1 + 2*k2 + 2*k3 + k4)
40
41 y = np.array([np.pi / 2, np.pi / 2, 0.0, 0.0])
42
43 t_max = 15.0
44 dt = 0.02
45 num_steps = int(t_max / dt)
46
47 x1, y1 = [], []
48 x2, y2 = [], []
49

```

Slika 1: Implementacija parametara i funkcija za izračun matrice inercije (metričkog tenzora) u Pythonu *$\mathbf{N}:\mathbf{B}$ matrica rotacije je element Liejeve strukture $SO(2)$ koja predstavlja skup 2×2 matrica sa svojstvom da je determinanta jednaka 1. U kontekstu rada izostavljeno je njeno svojstvo i detalji o samoj strukturi.*

U svakom vremenskom koraku simulacije računalni program koristi funkciju `numpy.linalg.solve(M, F)` kako bi izravno i stabilno izolirao čista kutna ubrzanja $\ddot{q}$ izbjegavajući invertiranje matrice koje je numerički nezahvalan.

Izolirana ubrzanja zatim se integriraju pomoću numeričke metode Runge-Kutta reda četiri (RK4) s vremenskim korakom $dt = 0.02$ s. Za razliku od Eulerove metode koja linearno divergira i umjetno stvara energiju, RK4 uzorkuje nagibe faza u četiri različite točke unutar koraka i računa njihov ponderirani prosjek, pružajući visoku lokalnu točnost reda $O(dt^5)$ i dugoročnu stabilnost zakona očuvanja energije.

```

49
50 for _ in range(num_steps):
51     t1, t2 = y[0], y[1]
52
53     R1 = so2_matrix(t1)
54     R2 = so2_matrix(t2)
55
56     p1 = R1 @ np.array([0, -L1])
57     p2 = p1 + R2 @ np.array([0, -L2])
58
59     x1.append(p1[0])
60     y1.append(p1[1])
61     x2.append(p2[0])
62     y2.append(p2[1])
63
64     y = rk4_step(y, dt)
65

```

Slika 2: Izračun derivacija i integracija RK4

3 Zaključak

U ovome radu uspješno je opisan i simuliran složeni problem gibanja dvostrukog njihala. Pokazano je da prelazak s koordinatne mehanike sila na Lagrangeov formalizam i geometrijsku interpretaciju konfiguracijskog prostora preko torusa pruža izuzetno elegantan i strukturan uvid u fiziku nelinearnih sustava. Korištenjem programskih alata u jeziku Python i robusnih numeričkih metoda poput RK4 integratora uz optimizacije unutar simulacije, razvijena je stabilna, brza i fizikalno vjerna simulacija koja zorno rekonstruira fascinantno ponašanje determinističkog kaosa u prirodi.

N.B: simulacija u mp4 formatu nalazi se na GitHub-u: <https://github.com/pavlarapic/PAF>

Literatura

- [1] C. Lessig, *A Primer on Geometric Mechanics*, Computational + Mathematical Sciences, Caltech, arXiv:1206.3302v1, 2012.
- [2] J. Awrejcewicz, D. Sendkowski, *Geometric Analysis of the Dynamics of a Double Pendulum*, Journal of Mechanics of Materials and Structures, Vol. 2, No. 8, 2007.
- [3] V. I. Arnold, *Mathematical Methods of Classical Mechanics*, Springer-Verlag, New York, 2nd edition, 1989.